CSSE 220 – Object-Oriented Software Development
Rose-Hulman Institute of Technology

Worksheet 19

Name (Print):_____ Section:_____

1. **Classify the mistake.** Write compile-time, runtime, or logic error.

```
// A                        // B
int score = "10";           int[] a = {4, 8};
                            System.out.println(a[2]);
// C: intended to find the mean of 4 and 8
int mean = 4 + 8 / 2;
```

A: _____ B: _____ C: _____
What exception does B produce? _____

2. **Trace the handler.** Write the output in order. Cross out the statement in the try block that is skipped. Which catch runs, and why?

```
System.out.println("Start");
try {
    int level = Integer.parseInt("two");
    System.out.println("Loaded");
} catch (NumberFormatException e) {
    System.out.println("Use default");
} catch (Exception e) {
    System.out.println("Other problem");
} finally {
    System.out.println("Cleanup");
}
System.out.println("Ready");
```

Output (one line per statement):

_____
_____
_____
_____

Which catch runs, and why? _____

3. **Throw or handle?** Fill the two blanks to reject a negative level and recover in the caller.

```
static void startLevel(int level) {
    if (level < 0) {
        ______ new IllegalArgumentException("Negative level");
    }
}
// In main:
try { startLevel(-1); }
catch (_________________________ e) {
    System.out.println("Choose another level");
}
```

Which method throws? Which method handles? _____
A method declares `throws FileNotFoundException`. What are the caller's two options?

_____

4. **Draw the chain.** Use the class from lecture, not `java.util.LinkedList`. Each Node stores `int value` and `Node next`; `new Node(v, n)` stores those two arguments. Starting empty, execute:

```
list.addAtBeginning(3);
list.addAtBeginning(5);
list.addAtBeginning(8);
```

Draw head, all three node boxes (value and next), arrows, and the final null.

5. **Follow the references.** For the list in Question 4, complete the traversal. List the successive values of `current.value`. Does head change?

```
int count = 0;
Node current = this.head;
while (____________________) {
    count++;
    current = ____________________;
}
return count;
```

Values: _____ Returned count: _____
Why is `current.next != null` incorrect? Consider both a nonempty list and an empty list.

6. **Change the front.** Complete both methods inside our custom list. Deleting from an empty list should do nothing.

```
public void addAtBeginning(int value) {
    Node newNode = new Node(value, ______________);
    this.head = ______________;
}
public void deleteFirst() {
    if (____________________) {
        this.head = ____________________;
    }
}
```

Starting again with Question 4's list, call `deleteFirst()` once. Draw the remaining chain. What is head after deleting the only node in a list?

7. **Compare costs.** Give the worst-case Big-O in terms of n, the number of elements.

Operation	Custom singly linked list	ArrayList
Read element at index		
Insert at front		
Remove first (nonempty)		

What is the cost of our list's `size()`? Why?